



ECE/CS 552: Cache Concepts

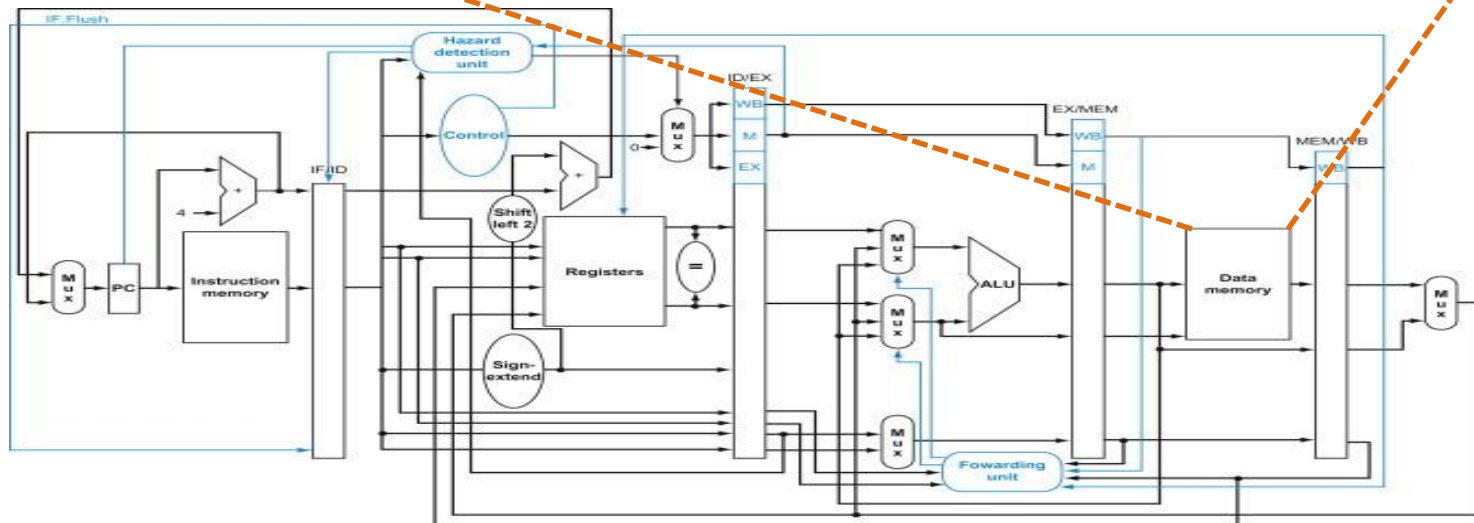
Boone Severson

Lecture notes based in part on slides created by
Joshua San Miguel, Mikko Lipasti, Mark Hill, David
Wood, Guri Sohi, John Shen and Jim Smith

Memory

Data is just an ocean of unlabeled ones and zeros that our program can load and store. This method of storing these bits has evolved over the decades, but the fundamentals remain.

011001001001010110100110101011111110001010
1010010111001001010101010101001010101010
110100001010110010001011110100101010010010
100001101001010010100100010010101001010010
110010101010010100010010101010111000000010
101011101010101011110100100010110100110101
001010101001010101010010011100101010010101



Types of Memory

Type	Size	Speed	Cost/bit
Registers	< 1KiB	< 1ns	\$\$\$\$
On-chip SRAM	8KiB – 300MiB	< 20ns	\$\$\$
Off-chip DRAM	64MiB – 1TiB	< 100ns	\$\$
Disk	32GB – 1PB	< 20ms	\$

- **Overview of Memory Types**

Our modern computers have many types of memories. The table summarizes the four general categories and gives you a sense of the differences in size, speed, and cost per bit.

- **On-Chip Registers and SRAM**

The on-chip registers are small, blazing fast, but the cost is high. SRAM is the acronym for static random access memory, which is used for structures like caches. Static means that once you write a value, the SRAM holds it as long as the chip is powered up.

- **SRAM Characteristics**

They are larger but slower than registers. Their size and speed are connected; the larger or more dense they get, the slower they get because wires get longer, and decode logic grows. SRAM sizes on processors continue to grow for reasons we'll explain in a bit.

- **Dynamic RAM (DRAM)**

DRAM or dynamic random access memory. The dynamic means that values are held in capacitors that leak their charge until they either get to be refreshed or they become zero.

- **Main Memory and HBM**

DRAM memory has been our main memory that stores almost all the program data in GPUs and ML accelerators. HBM or high bandwidth memory lives inside the multi-die package and consists of layers of DRAM silicon stacked to make a cube.

- **DRAM in Apple Silicon**

Apple silicon also puts DRAM inside the package, but generally, the largest and least expensive DRAM lives off the package and is connected via board traces to the processor.

- **Disk Storage**

Disk today consists either of spinning disk-shaped platters that hold magnetic charges or solid-state flash chips. This technology is inexpensive per bit and has capacities that can store all of your files reliably.

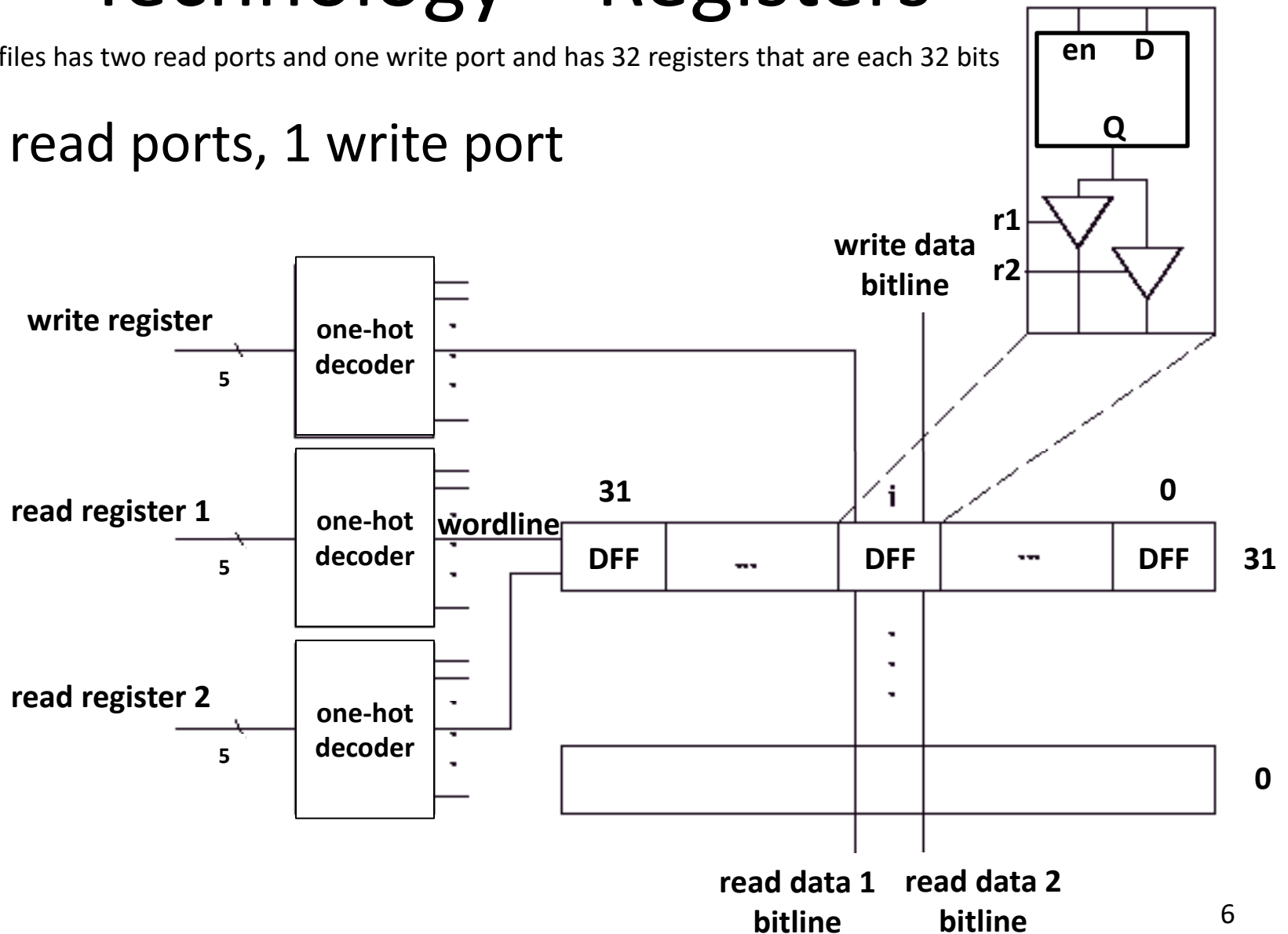
- **Speed of Disk Storage**

However, relative to the other storage, they're slow. You'll feel the speed when you launch large applications such as program data and graphics that are read from disk and put into DRAM.

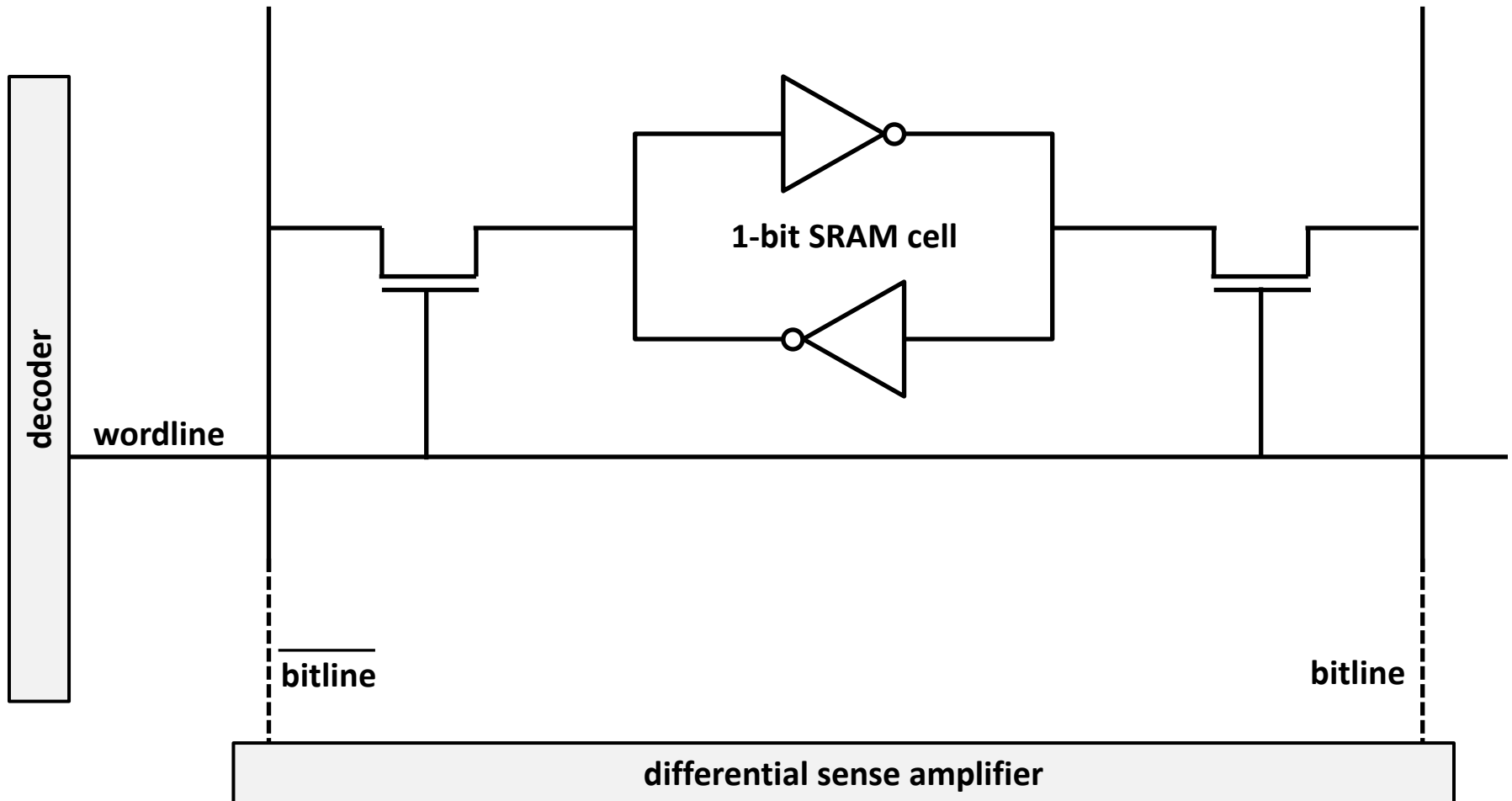
Technology – Registers

Register files has two read ports and one write port and has 32 registers that are each 32 bits wide.

- 2 read ports, 1 write port



Technology – SRAM



- **SRAM Technology**

SRAM technology is conceptually very similar to register files, but instead of a flip-flop storing each bit, it uses a level-sensitive latch to hold data.

- **SRAM Cell Structure**

In this simplified diagram, the cross-coupled inverters are actually two CMOS transistors, making this the classic six-transistor or 6T SRAM cell.

- **Array Structure of SRAM**

These cells are built into an array with a one-hot address decoder to choose the word line, which activates the word or row of the array by turning on the pass transistors or transmission gates.

- **Reading and Writing in SRAM**

The complement column bit lines are used to write or read from the corresponding bit position. When writing, the drive strength of the new value flowing down the top is enough to upset the latch and change its value.

- **Reading Large SRAM Arrays**

When reading large SRAM arrays, the bit line and complement bit line pass their values down to the bottom, where their voltage level has decreased substantially due to resistance.

- **Sense Amplifiers in SRAM**

They can't simply be read at normal logic levels, so these SRAMs will get a differential sense amplifier. A sense amp amplifies small voltage perturbations back to the appropriate logic level.

- **Low-Cost SRAM Devices**

In low-cost SRAM devices, we typically have one bit line per bit, so ground is the reference. For high-performance SRAM both the true and complement value are sent down, leveraging the voltage swings to quickly determine logic values.

Technology – DRAM

- Logically similar to SRAM
- Commodity DRAM chips
 - e.g., 1Gb per chip
 - Standardized address/data/control interfaces
- Very dense
 - Bit stored in capacitor
 - Decays over time: must *refresh*
 - Destructive read: must rewrite on read
- Density improving significantly over time
- Latency barely improving

- **Introduction to DRAM**

DRAM technology is logically similar to SRAM in addressing. Commodity DRAM chips are very inexpensive, with typical capacities of 1 Gb per chip.

- **Characteristics of DRAM**

They come with standardized data, address, control, interfaces and are very dense, requiring minimal logic per bit cell—just a transistor that provides access via the word line.

- **Data Storage in DRAM**

The data itself is stored as a charge in the capacitor, making up the one transistor, one capacitor (1T1C) element. The capacitor means charge decay occurs over time due to leakage and must be periodically refreshed by reading them out and by writing them back in.

- **DRAM Refresh Mechanism**

This decay necessitates periodic refreshing by reading them out and back in. While DRAM density has increased for cost reasons, latency has barely improved compared to the speed increase in computation logic.

Memory Requirements

- Need lots of bandwidth
 - e.g., 1 IPC, 1 GHz
 - 40% D-mem insns $\times$ 4B per D-mem insn
 - 100% insns $\times$ 4B per I-mem fetch
 - Thus required bandwidth = 5.6 GiB/s
- Need lots of storage
 - Ranging from 64MiB to multiple TiB
- Must be cheap per bit
- These requirements seem incompatible

- **Memory Requirements for Processors**

So, what are the memory requirements imposed by our processor and important applications? We need lots of bandwidth, lots of storage, and we need to ensure it's affordable.

- **Bandwidth Calculation**

Assuming a processor with one IPC running at 1 GHz, if 40% of our instructions access data memory, and all instructions need to fetch from I-MEM, we need 5.6 gigabytes per second of bandwidth to keep our processor from blocking.

- **Storage Needs**

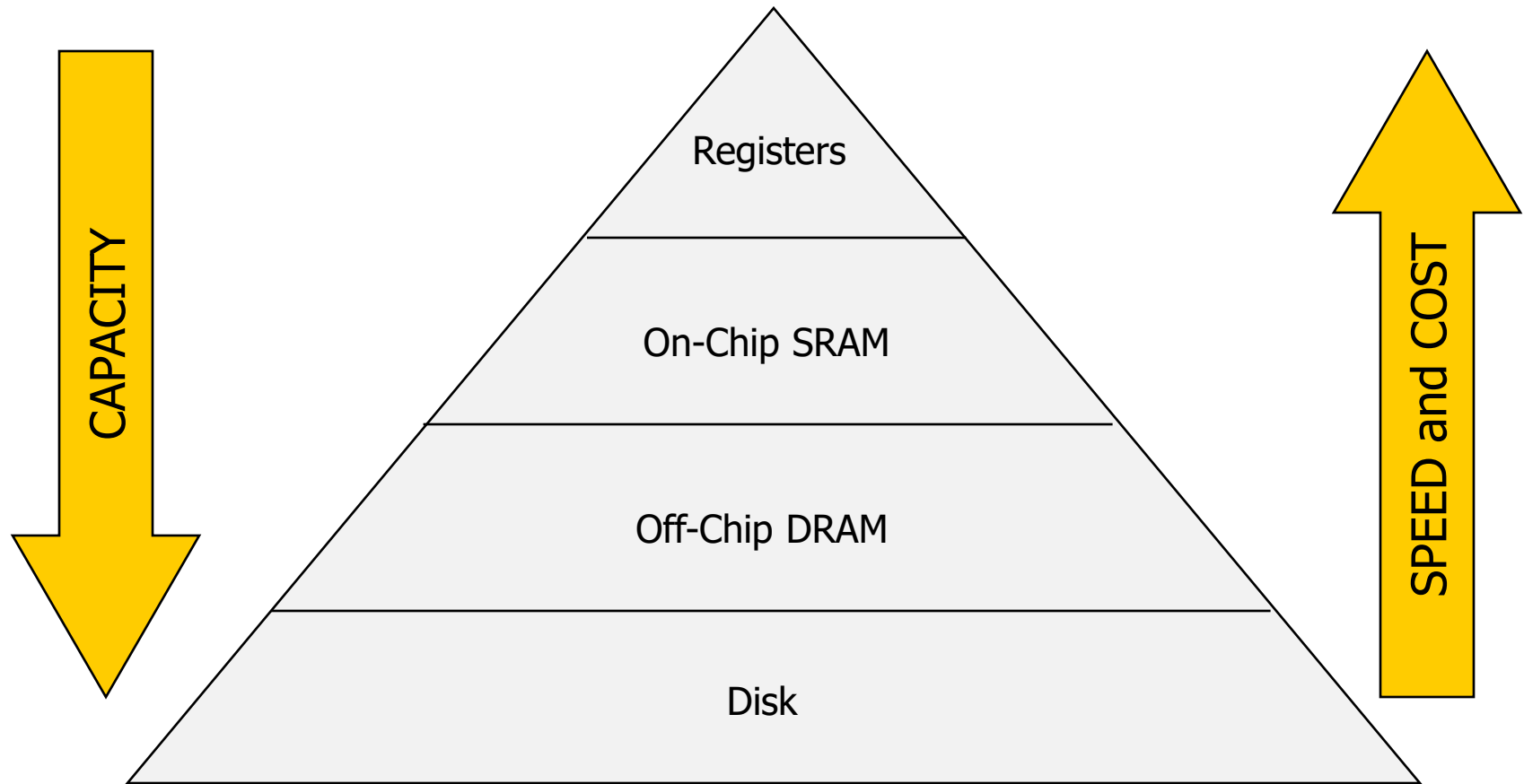
Even mobile applications require dozens of megabytes, while high-performance servers will hold multiple terabytes of memory.

- **Cost Considerations**

we need a per bit cost at a minimum. These requirements seem incompatible at first glance.

- What can we do?

Memory Hierarchy



- **Memory Hierarchy Solution**

The general solution is to form a memory hierarchy. You do this in your everyday life: registers are like front pockets – easily accessible and only fit what you need right now, On-Chip SRAM is your backpack – hold your laptop, books but takes a few extra seconds to open up, and off-chip DRAM is your apartment – which keeps all your books and clothes. Disk is the storage at the basement of the building.

- **Data Access and Cost**

The idea is that not all bits are created equal. We have lots of data we could access, but much of it is accessed less frequently. Different memory technologies offer trade-offs in speed, capacity, and cost. Registers and SRAM are more expensive but having small amount of them can really help speed up the accesses to critical data that we use often while the rest of the data can reside in DRAM or Disk.

Memory Hierarchy

Why memory hierarchy?

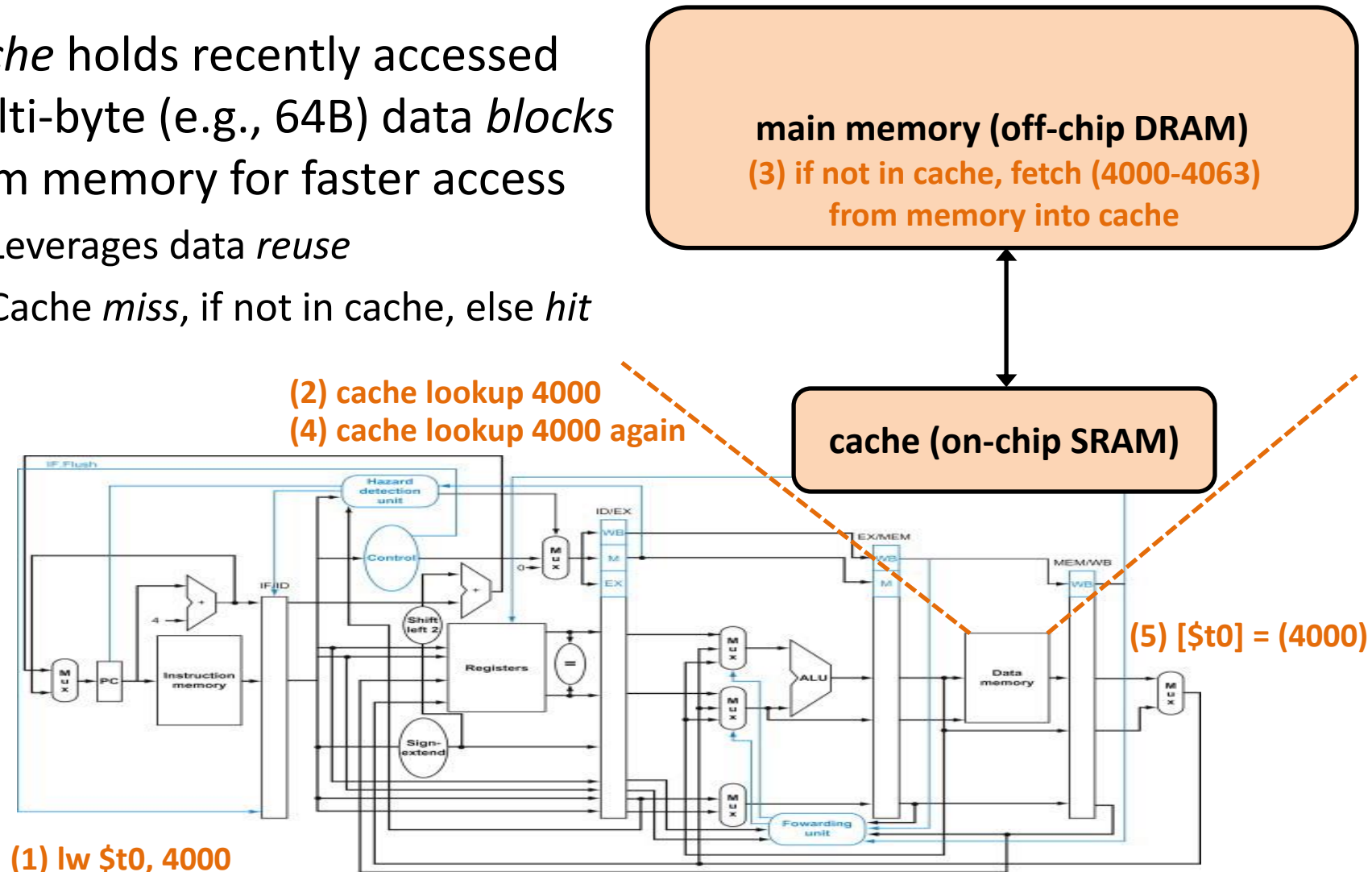
- Fast and small memories
 - Quick access (fast cycle time), expensive
 - For data that the program uses often
- Slow and large memories
 - High capacity, cheap
 - For the rest of the program data
- **Together:** provide appearance of large, fast memory with cost of cheap, slow memory

- So why a memory hierarchy because the heterogeneity of memory Technologies works very well with the heterogeneity of the data that our programs access fast memories are expensive but we can dedicate a small amount of them for storing data that we expect to use often, while slow memories are much cheaper and more dense which is what works to store the rest of our data taken together. This hierarchy appears to be a large fast memory with roughly the cost of slow memory.

Memory Hierarchy

Cache holds recently accessed multi-byte (e.g., 64B) data *blocks* from memory for faster access

- Leverages data *reuse*
- Cache *miss*, if not in cache, else *hit*



- **Cache Implementation**

Here we start our implementation of the memory hierarchy of our processor with a cache. On-chip caches are typically built using SRAM and sit between the register file and main DRAM memory.

- **Cache Functionality**

The cache aims to hold recently accessed data stored in the form of multi-byte blocks, often 64 bytes in size. So now Cache blocks and cache lines are used synonymously.

- **Cache Lookup Process**

Caches leverage data reuse. They often offers faster access than main memory so it behooves us to reserve cash storage for data we expect to use a lot. When a load or store instruction hits the memory stage, we perform a lookup in the cache. If the data at address 4000 is in the cache, it's a cache hit.

- **Cache Miss Handling**

If not, it's a cache miss, and we go to main memory to fetch it. We don't just fetch 4 bytes; we fetch the 64-byte cache line at address 4000 or byte 4000 through 4063.

- **Writing Cache Data**

This cache line is written into the cache, and we replay the lookup, allowing us to read out the four bytes at address 4000 and put it into register t0 during the write back stage.

Memory Hierarchy

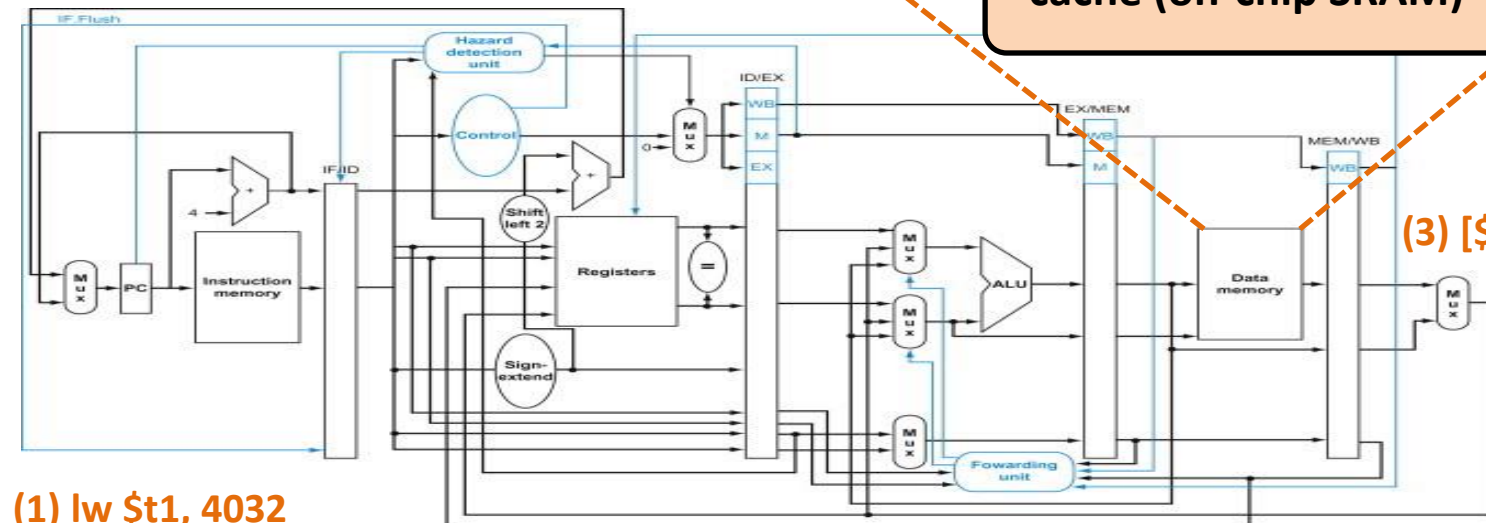
Cache holds recently accessed multi-byte (e.g., 64B) data *blocks* from memory for faster access

- Leverages data *reuse*
- Cache *miss*, if not in cache, else *hit*

main memory (off-chip DRAM)

cache (on-chip SRAM)

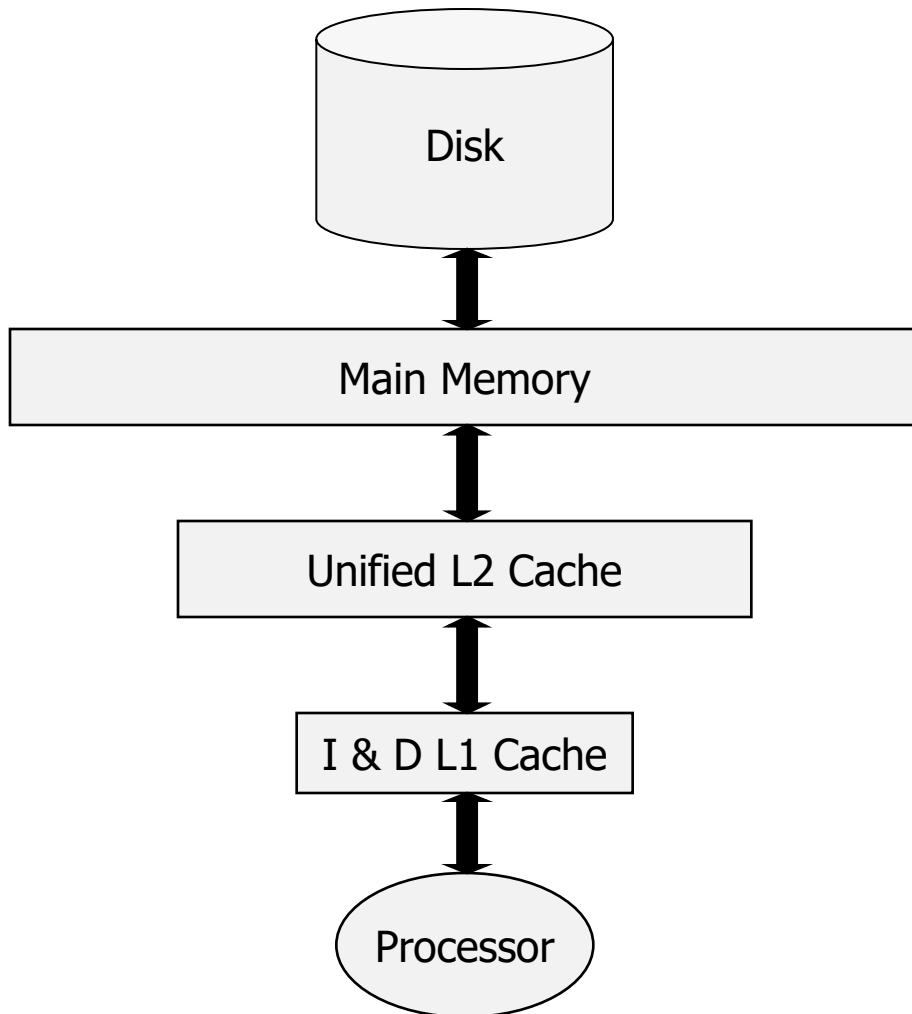
(2) cache lookup 4032, hit



- **Subsequent Load Instruction**

If another load instruction reads address 432, we check the cache. This word exists in the cache because it's within the 64-byte block fetched earlier, allowing immediate delivery to the register file in one cycle

Memory Hierarchy



- *Unified* cache:
 - Contains both insns and data
 - Main memory is usually unified
- Inclusive vs. exclusive L2:
 - *Inclusive*: a block X in L1 must also be in L2
 - *Exclusive*: a block X in L1 must not be in L2

- **Programming for Cache Efficiency**

Programmers looking for optimal performance should ensure that blocks of related data, like structs, are tightly packed and cache line aligned to maximize cache use.

- **Multi-Level Cache Structure**

A modern memory hierarchy consists of multiple levels of on-chip caches. The first level cache (L1) is closest to the processor, sized for speed, and typically split into instruction and data caches.

- **Upper-Level Caches**

Upper-level caches like L2 are often unified, storing blocks from both instruction and data memory. Main memory is usually unified as well.

- **Cache Inclusivity**

Upper-level caches can be inclusive or exclusive. An inclusive L2 cache contains a superset of L1 blocks, while an exclusive L2 cache ensures that no block is in both caches simultaneously.

Memory Hierarchy

- Data locality (i.e., reuse):
 - *Temporal* locality
 - Programs access same memory location repeatedly
 - *Spatial* locality
 - Programs access nearby locations around the same time
- Empirically observed
 - Significant!
 - Even small 8KB cache often satisfies >90% of accesses to a multi-MB dataset

- **Data Locality Concepts**

The main idea for caches is data locality or reuse. There are two flavors: temporal and spatial. Temporal locality means applications frequently access the same data over time.

- **Spatial Locality**

Spatial locality refers to accessing nearby memory locations. These localities are common in most programs due to the nature of information processing.

- **Small Cache Efficiency**

Even a small cache of 8 kilobytes can satisfy over 90% of all accesses in a multi-terabyte dataset.

Why Locality?

- Analogy:
 - Library/bookstore (disk)
 - Bookshelf (main memory)
 - Stack of books on desk (cache)
 - Opened book on desk (register)
- Likelihood of:
 - Referring to same book or chapter again?
 - Probability decays over time
 - Book moves to bottom of stack, then bookshelf, then library
 - Referring to chapter $n+1$ if looking at chapter n ?

- **Book Storage Analogy**

To illustrate locality, consider how you store books. While all books can be found in a library, you keep those needed for your major on a shelf, like your working set, with frequently accessed ones on your desk.

- **Locality in Action**

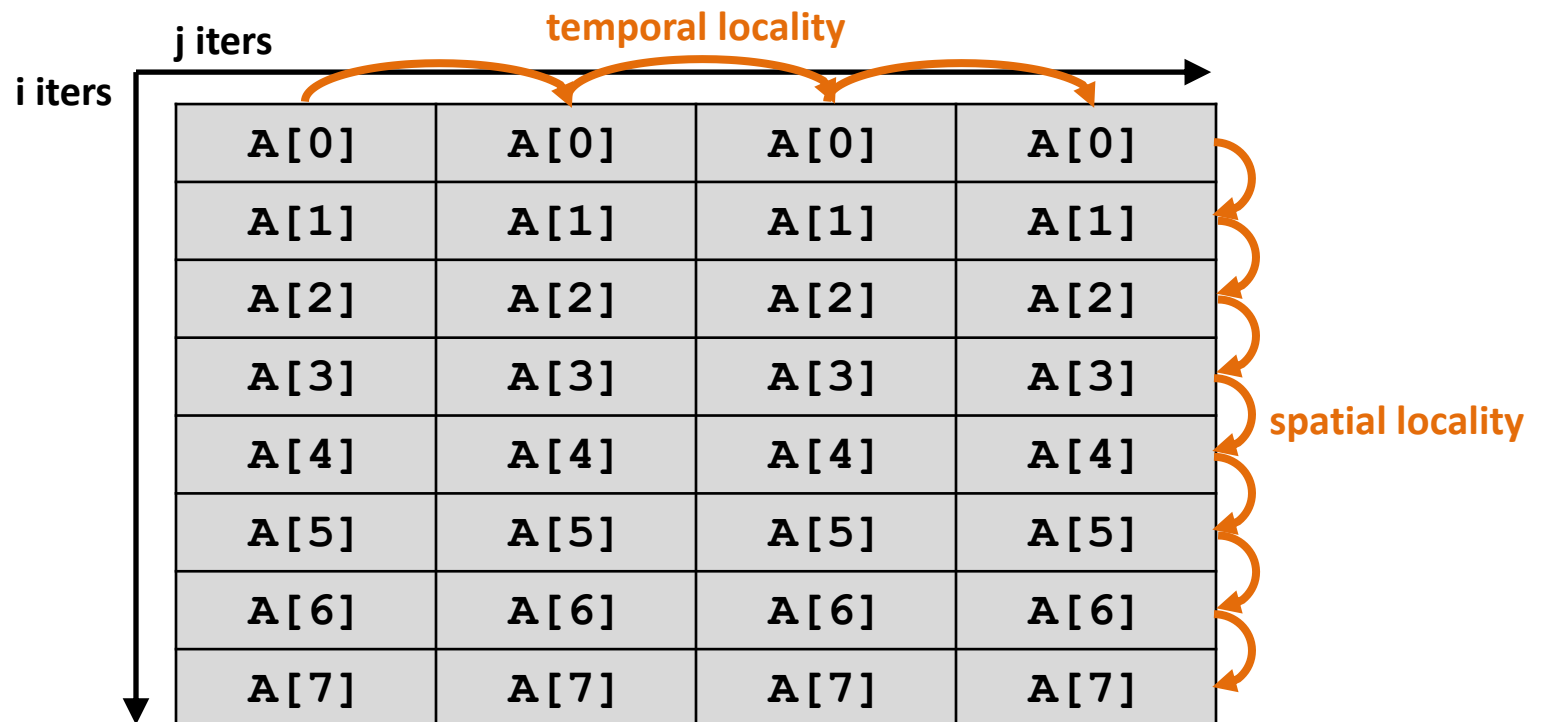
As you study, you repeatedly access the same books (temporal locality) and frequently move to the next chapter (spatial locality).

- **Array Access Patterns**

In a program using nested loops, temporal locality exists in each row, while spatial locality appears across iterations as neighboring elements are accessed.

Locality

```
int A[N];  
for (int i = 0; i < N; i++)  
    for (int j = 0; j < 4; j++)  
        ... = A[i];
```



Caches

Cache architecture consists of:

- *Data array*: Array of multi-byte blocks of memory contents
- *Tag array*: Array of tags (i.e., specific address bits) that uniquely identify each block
 - Also contains metadata:
 - *Valid* bit: specifies that cache location is actually occupied
 - *Dirty* bit: specifies that cache block is modified and needs to be written back to memory (discuss later)
 - *Replacement* bits: specify ranking of blocks that are likely to be used again (discuss later)

- **Cache Architecture Overview**

Cache architecture consists of two key structures: a data array that holds cache line blocks and a tag array that stores tags for corresponding data blocks.

- **Tags and Metadata**

A tag is a unique identifier for each block, containing a valid bit, a dirty bit, and replacement bits for ranking based on reuse likelihood.

Caches

Placement:

Where can a block of memory go?

Replacement:

How do we make space for new blocks?

Write Policy:

How do we propagate modifications to memory?

Cache Design Considerations

Key design considerations for building a cache include placement (where to store blocks), replacement (how to make space for new blocks), and write policy (how to propagate modifications).

Caches – Placement

- Since memory address range exceeds cache capacity, map memory addresses into finite set of cache storage locations.
- *Direct-mapped* cache:
 - Each memory address maps to a single location (*index*) in the cache
 - Will discuss *set-associative* caches later

- **Direct Mapped Cache**

The goal of cache placement is to map memory addresses to finite cache locations. Direct-mapped caches allow only one place for a block based on memory address.

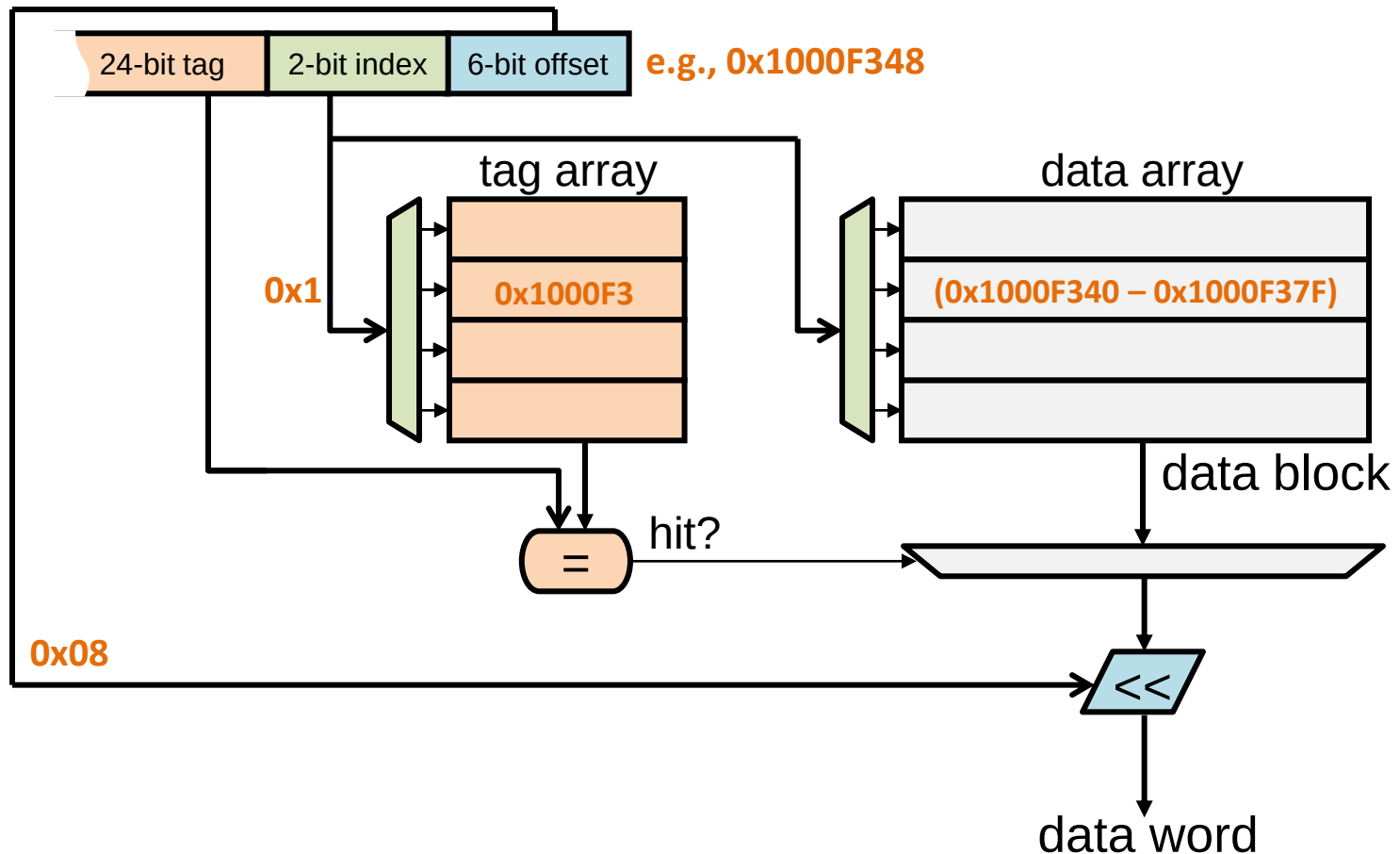
- **Cache Indexing Scheme**

The standard indexing scheme splits the memory address into tag bits, index bits, and block offset bits.

Caches – Placement

Memory address = { tag, index, block offset }

- e.g., 256B cache capacity (can hold four 64B blocks)



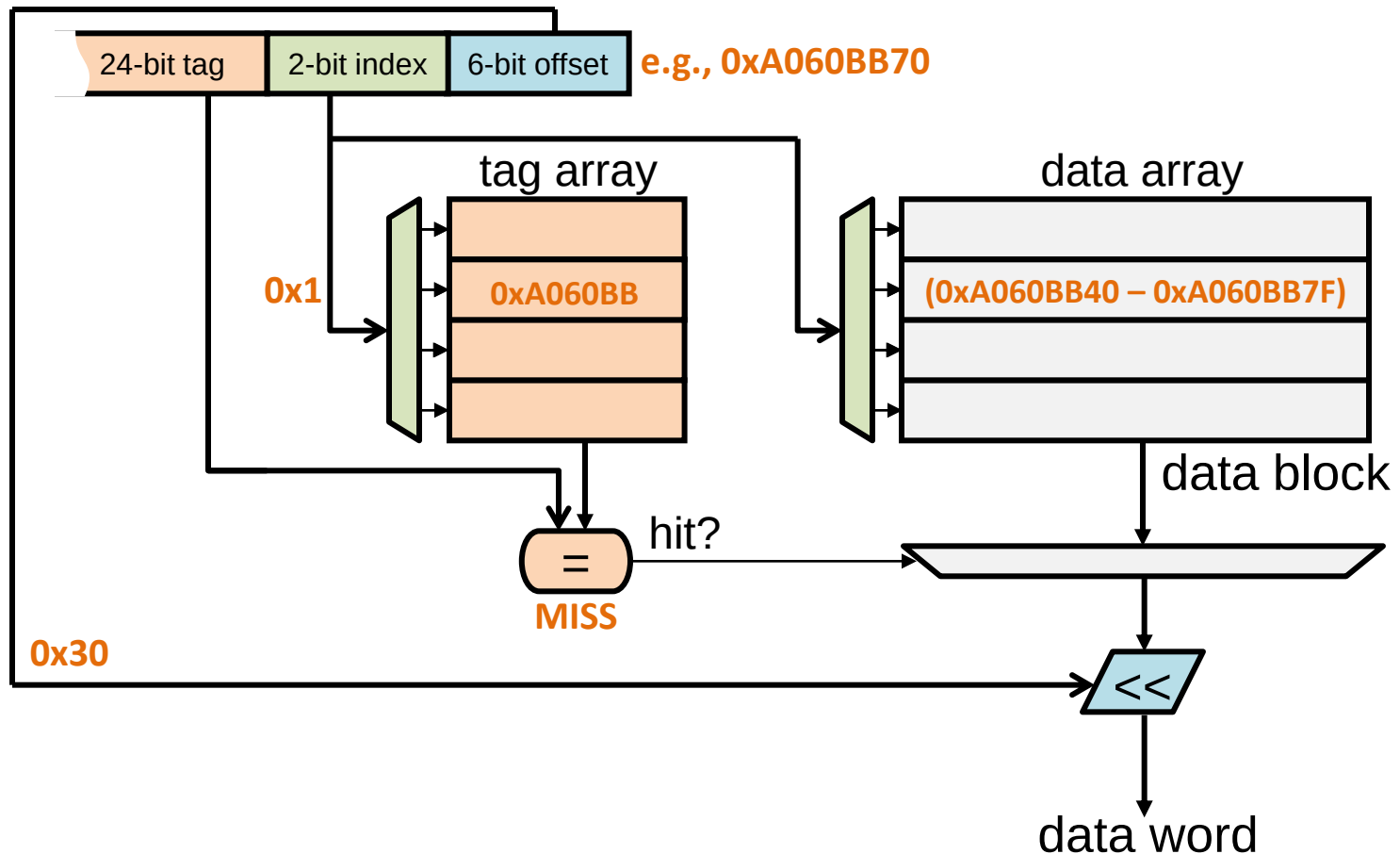
Caches – Replacement

- Since cache capacity is finite, may need to *evict* a block in the cache to make space for another one.
- Direct-mapped cache:
 - Straightforward, evict current block at index

Caches – Replacement

Memory address = { tag, index, block offset }

- e.g., 256B cache capacity (can hold four 64B blocks)



Caches – Write Policy

Since multiple copies of a block can exist in the memory hierarchy, correctly propagate any modified values to the next level.

Write-Allocate: Upon a store, insert the block (and its modifications) in the cache.

Write-No-Allocate: Upon a store, don't occupy cache space for the block.

Write-Through: Propagate modifications directly to the next level on every store (typically with write-no-allocate).

Write-Back: Propagate modifications to the next level on eviction (typically with write-allocate). Requires *dirty* (i.e., modified) bit.

Caches – Example

- e.g., Given 8-bit memory addresses, 32B cache with 4B blocks

tag (3 bits)	index (3 bits)	2 bits
--------------	----------------	--------

– Memory addresses shown in base 4

Cache Contents (index 0 -> index 7)	Request	Outcome
0000, 0010, 0020, 0030, 0100, 0110, 0120, 0130	3020	Miss
0000, 0010, 3020 , 0030, 0100, 0110, 0120, 0130	3030	Miss
0000, 0010, 3020, 3030 , 0100, 0110, 0120, 0130	2100	Miss
0000, 0010, 3020, 3030, 2100 , 0110, 0120, 0130	0012	Hit
0000, 0010, 3020, 3030, 2100, 0110, 0120, 0130	0020	Miss
0000, 0010, 0020 , 3030, 2100, 0110, 0120, 0130	0030	Miss
0000, 0010, 0020, 0030 , 2100, 0110, 0120, 0130	0110	Hit
0000, 0010, 0020, 0030, 2100, 0110, 0120, 0130	0100	Miss
0000, 1010, 0020, 0030, 0100 , 0110, 0120, 0130	2100	Miss
1000, 1010, 0020, 0030, 2100 , 0110, 0120, 0130	3020	Miss

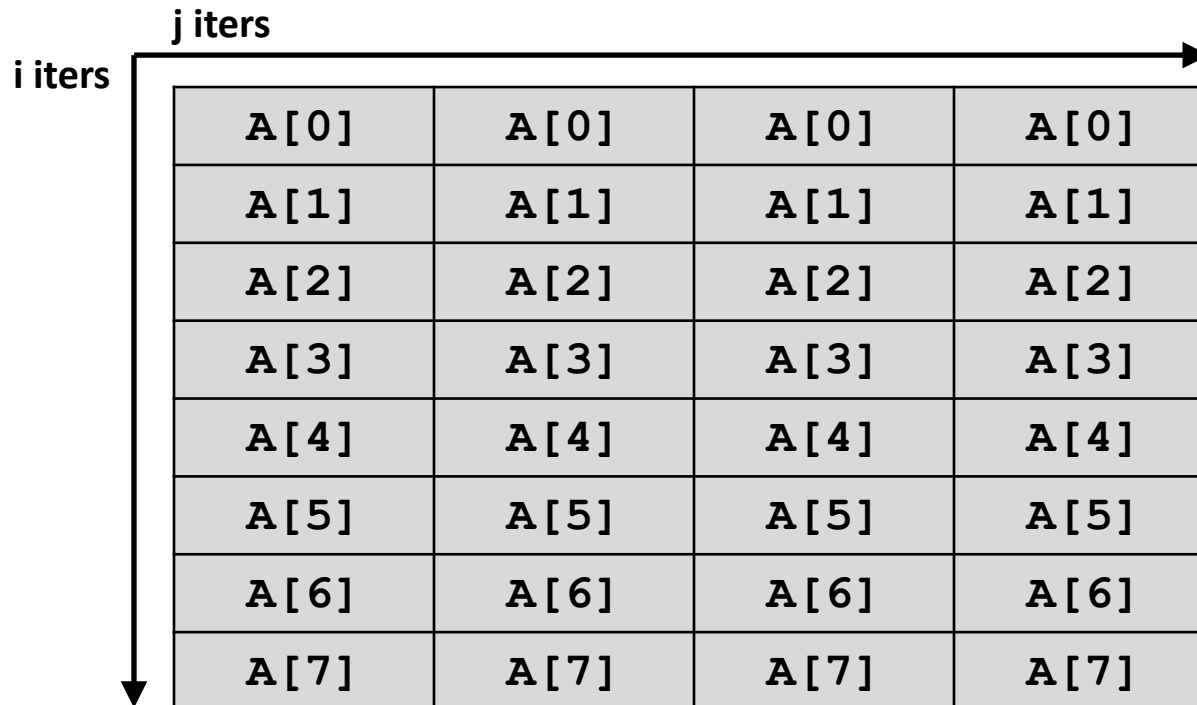
Exercise

- Given 32-bit memory addresses and 128-byte cache blocks, what is the minimum number of tag bits needed in a 64KB direct-mapped cache?
 - **$64\text{KB} / 128\text{B} = 512$ blocks in the cache**
 - **$32 \text{ bits} - \log(512 \text{ blocks}) - \log(128\text{B per block})$**
 - **Thus 16 tag bits**

Exercise

```
int A[256]; // &A[0] = 4000
for (int i = 0; i < 256; i++)
    for (int j = 0; j < 4; j++)
        ... = A[i];
```

Given 16-byte blocks,
how many cache hits due
to spatial locality?



Exercise

```
int A[256]; // &A[0] = 4000
for (int i = 0; i < 256; i++)
    for (int j = 0; j < 4; j++)
        ... = A[i];
```

Given 16-byte blocks,
how many cache hits due
to spatial locality?

3 × 64 = 192 hits

